

SECURITY ASSESSMENT REPORT

QA & Penetration Testing: Methodology and Evidence

A record of every check performed against the AI Jazeera ERP, how each one was carried out, and what it returned. Written to be reproducible: each technique lists the actual method used, so any result in this document can be re-run and confirmed independently.

Field	Detail
Target	AI Jazeera ERP - Express API (~6,000 LOC) + React SPA (~9,000 LOC)
Environment	localhost only (API :5000, Vite :5173) against the live MongoDB Atlas database
Date	9 August 2026
Authorisation	Owner-requested assessment of their own application
Checks executed	46 across 5 phases
Result	2 Critical, 1 High, 4 Medium, 9 Low. 12 attack classes tested and held.

Rules of engagement

Constraints adopted before testing began, and held throughout:

Rule	Why
No remote host was ever contacted. All traffic went to localhost .	The production deployment was out of scope. Findings about it are marked as needing the owner's verification.
No destructive testing. No flooding, no DoS.	Rate limits were confirmed by reading <code>RateLimit</code> response headers, not by exhausting them.
No real employee documents were downloaded.	The document-exposure finding was proven with a synthetic file the tester uploaded, never with a real passport scan.
Test data only, fully removed afterwards.	A throwaway admin, a worker, test employees and synthetic files. All deleted and verified gone.
Audit-log entries were left in place.	Tampering with an audit trail is worse than leaving noise in it. The rows are attributed to the test account.

1. Methodology

The assessment ran in five phases. Order matters: reading the code first tells you where to aim, and attacking blind wastes effort on defences that are obviously present. Equally, code review alone proves nothing - several findings here looked fine on the page and only failed when actually executed.

Phase	What it covers	How
1. Static review	Auth and session logic, RBAC, injection defences, output escaping, upload handling, error paths, secrets	Line-by-line reading of every route, controller, service and middleware; targeted grep sweeps for dangerous patterns
2. Dependency & config audit	Known CVEs, environment hygiene, secret leakage	npm audit, git history scan, env validation review
3. Penetration test	Whether the defences actually hold under attack	Live HTTP attacks with curl against a running server, using a seeded throwaway admin
4. QA / functional	Does the product work: flows, states, responsiveness, console health	Driving the real React app in a browser, plus DOM and network inspection
5. Verification	Did the fixes actually fix it	Re-running the exact attacks that succeeded, plus regression checks

Reading the result column

Marker	Meaning
BLOCKED	The attack was attempted and the application correctly rejected it.
VULNERABLE	The attack succeeded. Evidence recorded.
NOTED	Works as built, but carries risk worth recording.
CLEAN	Inspection found nothing wrong.

Attack surface mapped before testing

Surface	Exposure	Routes
/api/auth/*	Unauthenticated	login, refresh, logout
/api/{employees, clients, deployments, attendance, documents, quotations, dashboard}	Auth + staff-only	Full CRUD, role-gated writes
/api/nfc/*	Auth + Admin only	30+ routes incl. uploads, CSV, QR
/api/timesheet-processor	Auth + Admin	Parses uploaded .xls
/c/:token	Public, no auth	Server-rendered HTML card page
/c/:token/e	Public, no auth, WRITE	Analytics beacon
/nfc-media/:filename	Public, no auth	Static media
/api/health	Public	Liveness

2. Penetration testing - checks performed

46 checks, grouped by attack class. Every row was executed against a running server. The method column states the actual technique so the result can be reproduced.

2.1 Authentication and session management

#	Check	Method	Result
1	Unauthenticated access to a protected route	GET /api/employees with no Authorization header	BLOCKED 401
2	JWT alg:none forgery	Hand-built token: base64url header {"alg":"none"} + admin payload + empty signature	BLOCKED 401
3	Signature tampering	Valid token with a character appended to the signature	BLOCKED 401
4	Expired access token	Reused a token past its 15-minute TTL	BLOCKED 401
5	Login brute-force throttling	Read RateLimit headers on POST /api/auth/login (non-destructive)	30 / 15 min
6	General API throttling	Read RateLimit-Policy on /api	600 / 15 min
7	Refresh-cookie flags	Inspected the cookie jar after login	HttpOnly, path-scoped
8	Token theft via XSS	In-page JS probe of localStorage, sessionStorage and document.cookie	ALL EMPTY
9	Username enumeration	Code review of the login path: dummy bcrypt comparison for unknown emails, identical 401 for all three failure modes	CLEAN
10	Refresh-token rotation and reuse detection	Code review of rotation, 30s grace window and theft response	CLEAN

2.2 Authorization and privilege escalation

#	Check	Method	Result
11	Worker isolation (the P2-M1 boundary)	Provisioned a real Worker login, then requested all 10 staff modules with its token	403 on all 10
12	Mass assignment / privilege injection	POST /api/employees with extra role, _id, isActive keys in the body	STRIPPED
13	Role-guard coverage	Enumerated every router; confirmed requireAuth + requireStaff + per-verb role guards	CLEAN
14	Admin-only module gating	Requested /api/audit, /api/nfc, /api/timesheet-processor as non-Admin	BLOCKED 403

2.3 Injection

#	Check	Method	Result
15	NoSQL injection - authentication bypass	POST /api/auth/login with {"email":{"\$ne":null}, "password":{"\$ne":null}}	BLOCKED 400
16	NoSQL operator injection via query string	GET /api/employees?search[\$ne]=x (URL-encoded)	BLOCKED 400
17	Regex injection / ReDoS	Reviewed all 5 services building \$regex filters for input escaping	CLEAN
18	Path traversal on public media	6 payloads against /nfc-media/: . . /, URL-encoded, double-encoded, backslash, dot-slash variants	404 on all 6

#	Check	Method	Result
19	vCard property injection	Set a bio containing newlines plus forged TEL : / EMAIL : lines, then fetched the .vcf	ESCAPED
20	HTTP header injection via filename	Inspected Content-Disposition on the vCard download	SANITISED

2.4 Cross-site scripting

The public NFC card page is server-rendered HTML built by string concatenation, which is the highest-risk XSS surface in the codebase. Six payloads were stored through the authenticated API, then the resulting public page was fetched and parsed.

```
company.companyName ZZ</title><script>alert(1)</script>
company.website javascript:alert(document.domain)
company.mapLink javascript:alert(2)
company.address "><img src=x onerror=alert(3)>
employee.name ZZ<script>alert('xss')</script>
employee.jobTitle "><svg onload=alert(4)>
employee.linkedin javascript:alert(5)
employee.bio </p><script>alert(6)</script><p>
```

#	Check	Method	Result
21	Stored XSS on the public card page	Parsed the rendered HTML: counted real (unescaped) tags rather than grepping raw strings	0 injected tags
22	Script-tag and nonce integrity	Counted <script> occurrences and checked each carries the CSP nonce	1 tag, nonced
23	Inline event handlers	Regex for on*= attributes on real tags; 2 hits manually inspected	Both false positives
24	javascript: URL scheme	Checked how the stored javascript: URLs rendered into hrefs	Neutralised
25	Content-Security-Policy	Read the CSP header on the public page	Per-response nonce
26	XSS in the React SPA	Loaded the payload-laden records in the admin UI and inspected the DOM	Literal text

Note on check 23. An initial grep appeared to show surviving onload= handlers. Parsing the DOM properly showed both matches sat inside quoted attribute values where the quote had already been escaped to " - harmless. The naive grep was the bug, not the application. Recorded here because a string search over HTML will produce false XSS positives.

2.5 Transport, headers and public endpoints

#	Check	Method	Result
27	CORS policy against a hostile origin	OPTIONS preflight with Origin: https://evil.example.com	Not reflected
28	Security headers	Inspected all response headers	Full helmet set
29	NFC token enumeration	Requested 5 invalid/malformed tokens and compared status and body size	Identical 404s
30	Unauthenticated analytics writes	POST /c:/token/e three times with no credentials	204 accepted
31	Health-endpoint disclosure	GET /api/health unauthenticated	Leaks env + DB state
32	Error-response leakage	Triggered a 401 and read the body	Stack trace (dev only)

2.6 File upload and storage - where the Critical findings were

Everything above held. This section is where the assessment found real problems. The documents module stores passports, visas, iqamas, medical records and contracts, so it received the most attention.

#	Check	Method	Result
33	Public readability of an uploaded document	Uploaded a synthetic file through the real API as a Passport, then fetched the returned storage URL with no token, no cookie and no session	VULNERABLE - HTTP 200
34	URL leakage via redirect	Inspected response headers of GET /api/documents/:id/file	VULNERABLE - 302 Location
35	Does deletion actually delete? (raw files)	Uploaded a PDF-class file, deleted it via the API, then re-fetched the storage URL	VULNERABLE - still 200
36	Does deletion delete? (image files)	Same sequence with a PNG	Purged correctly
37	Root cause isolation	Called the storage SDK directly both ways to prove the difference	Confirmed
38	MIME-type spoofing	Uploaded HTML containing a script tag, declared as a Word document	VULNERABLE - 201
39	Malformed-file handling	Uploaded a file with a valid MIME label but corrupt content	500, not 400
40	Orphaned files on failed validation	Code review: cleanup calls fs.unlink on what is now a URL	Permanent orphan
41	NFC image upload (comparison)	Reviewed the separate NFC upload path	Format-validated

Evidence: check 33 - the file is public

```
POST /api/documents -> HTTP 201
stored URL: https://res.cloudinary.com/<cloud>/image/upload/v.../crm-documents/<id>.png
curl <that URL> (no auth whatsoever) -> HTTP 200 | bytes=70 | image/png
GET /api/documents/<id>/file -> HTTP 302
Location: https://res.cloudinary.com/<cloud>/image/upload/v.../crm-documents/<id>.png
```

The entire guard chain protects the redirect, not the bytes. Once the URL exists it is a permanent, unauthenticated capability that survives role changes, password changes and offboarding.

Evidence: checks 35 and 37 - deletion silently fails

```
1. public fetch BEFORE delete -> HTTP 200 (content served)
2. DELETE /api/documents/<id> -> HTTP 200 {"message":"Document deleted."}
3. GET /api/documents/<id> -> HTTP 404 (gone from the ERP)
4. public fetch AFTER delete -> HTTP 200 (STILL SERVING THE FILE)

Root cause, proven by calling the SDK directly:
destroy(id, { resource_type: 'raw' }) -> {"result":"ok"}
destroy(id) // what the app did -> {"result":"not found"}
```

The SDK *resolves normally* when it deletes nothing, so the surrounding try/catch never fired and the API reported success. Images deleted correctly, which is precisely why the bug survived review - it only affects PDF, Word and Excel, which is most of what an HR department uploads.

3. Dependency and configuration audit

#	Check	Method	Result
42	Known vulnerabilities in dependencies	npm audit --omit=dev on the server	1 high, 2 moderate
43	Secrets ever committed to git	Scanned full history for added env/secret files, plus a pattern sweep for connection strings and API keys across tracked files	CLEAN - never committed
44	Secret strength	Measured JWT secret lengths without printing values	128 chars each
45	Environment validation	Reviewed boot-time env validation and defaults	Fails fast; 1 risky default
46	Network binding	Checked listening sockets	0.0.0.0 - LAN reachable

The dependency findings are transitive and low-exposure for this application: a denial-of-service issue in a glob-expansion package, and a buffer-bounds issue in a UUID library reached through the spreadsheet exporter. The high-severity one clears with `npm audit fix`; the moderate one would force a breaking downgrade of the exporter and is not worth it.

4. QA - functional and interface testing

Security testing asks whether the app can be broken into. QA asks whether it works. These were run against the real React application in a browser, not against the API.

Area	Method	Result
Authentication flow	Signed in through the real form with seeded credentials	Works
Dashboard rendering	Verified stats, workforce/quotation breakdowns, expiring documents and activity feed all populated from live data	Works
Module navigation	Walked all 9 modules: dashboard, employees, clients, deployments, attendance, documents, quotations, timesheet processor, NFC	All render
Responsive layout	Scripted sweep at 375x812, measuring scrollwidth against viewport width on every page	0 overflow on 9/9
Console health	Captured all console output across the walk	No errors
Document preview	Opened the preview modal and confirmed it loads a blob URL through the authenticated client	Works
Data-layer integrity	Confirmed the SPA renders stored XSS payloads as literal text	Escaped

The only console output was a React Router v7 future-flag warning and one expected 401 per page load - the session-restore refresh call firing before login. Neither is a defect.

5. Remediation verification

After the two Critical findings were fixed, the exact attacks that had succeeded were re-run. Re-testing the original attack is the only evidence that counts; confirming the code looks better is not verification.

CHECK 33	guess public URL /raw/upload/<id>	-> 404 (was 200)
	guess public URL /image/upload/<id>	-> 404
	unsigned auth URL /raw/authenticated/<id>	-> 401
CHECK 34	GET /file (authorized)	-> 200, streamed, NO Location header
	GET /file (no token)	-> 401
	API response contains a storage URL?	-> NO
CHECK 35	upload -> DELETE -> list storage account	-> 0 objects (was: still served)
	multi-version doc, 2 files, DELETE	-> 0 objects (both purged)
CHECK 40	upload that fails validation after upload	-> 0 orphans left

Regression checks run alongside

Check	Method	Result
Legacy local-disk documents still work	Downloaded both pre-existing real documents (bytes discarded, only status and size read)	200, correct size
Version-specific download	Created a 2-version document and fetched each version explicitly	Correct bytes each
Browser preview still works	Opened the preview modal on a newly-stored document	blob: URL loads
No storage URL reaches the browser	Inspected all network requests made by the page	0 CDN requests

6. Summary of findings

Severity	Finding	Status
CRITICAL	Uploaded documents publicly readable with no authentication	FIXED
CRITICAL	Deletion does not delete; PDF/Word/Excel stay public forever	FIXED
HIGH	File-type allowlist bypassable via MIME spoofing	Open
MEDIUM	Corrupt upload returns 500 instead of 400	Reduced
MEDIUM	Rejected uploads leave permanent orphaned files	FIXED
MEDIUM	Card URL host may point at the wrong deployment	Needs owner check
MEDIUM	Legacy documents on ephemeral disk may already be lost	Needs owner check
LOW x9	Info disclosure, dependency CVEs, operational hardening	1 resolved, 8 open

7. Limitations of this assessment

Stated plainly, because an assessment that hides its gaps is worse than no assessment:

Not covered	Why
The production deployment	Only localhost was tested. Findings that depend on production configuration are marked as requiring the owner's confirmation rather than asserted.
Denial of service	Out of scope by the rules of engagement. Rate limiting was verified by reading headers, not by exhausting budgets.
Automated test suite	The project does not use one, and none was added. All verification here is manual and reproducible.

Not covered	Why
Business-logic abuse beyond money math	Quotation totals were verified as server-recomputed. Deeper workflow abuse (attendance fraud, deployment manipulation) was not modelled.
Infrastructure and platform	Hosting configuration, TLS setup, database network rules and CDN account settings were not assessed.
Social engineering and physical access	Out of scope.

Full technical detail, root-cause analysis and remediation guidance for every finding is in docs/SECURITY-AUDIT.md.